

MiniSQL Relational Database - Project Documentation

1. Introduction

This document provides a comprehensive architectural deep dive and documentation for the MiniSQL Relational Database built from scratch using C++17 and Qt6. The database implements a complete SQL pipeline from parsing to execution, backed by a persistent BTree-based storage engine.

2. Architectural Overview

The MiniSQL database is composed of several critical layers:

1. **Frontend / SQL Parsing:** Lexer, Parser, Abstract Syntax Tree (AST).
2. **Execution Engine:** Evaluator, Executor.
3. **Storage Engine:** BTree Indexing, Pager (Buffer Pool Manager).
4. **Utility / Framework:** Qt6 integrated components (QString, QVector, QFile).

3. Component Deep Dives

3.1 Lexer and Parser

Lexer: Tokenizes the raw SQL string into a stream of tokens. It handles strings, identifiers, numeric literals, and SQL keywords.

Parser: Consumes the tokens to build an Abstract Syntax Tree (AST). It uses a recursive descent parsing technique.

3.2 Abstract Syntax Tree (AST)

The AST represents the hierarchical syntactic structure of the SQL query. Nodes in the AST represent operations like `SelectStmt`, `InsertStmt`, `BinaryExpression`, etc.

3.3 Evaluator and Executor

Evaluator: Traverses the AST to validate semantics (e.g., table existence, type checking).

Executor: Executes the physical plan by interacting with the storage engine.

3.4 Pager (Buffer Pool Manager)

The Pager maps memory pages to disk blocks. It minimizes disk I/O by caching frequently accessed pages in memory using an LRU eviction policy.

3.5 BTree Implementation

The core indexing data structure. It provides logarithmic time complexity for insertions, deletions, and lookups.

4. Internal Workings

When a query like `SELECT * FROM users WHERE id = 5;` is executed:

1. The string is tokenized.
2. The parser builds a `SelectStmt` AST node.
3. The Evaluator ensures `users` exists and `id` is a valid column.
4. The Executor queries the BTree storage engine via the Pager to fetch the relevant tuples.

5. 50 Interview Q&A

1. **Q:** What is the purpose of the Lexer? **A:** To convert raw SQL strings into actionable tokens.
2. **Q:** How does the Parser handle operator precedence? **A:** By implementing a Pratt parser or recursive descent parsing with precedence levels.
3. **Q:** What is the role of the AST? **A:** It provides a structured representation of the SQL query for the evaluator.
4. **Q:** Why use C++17? **A:** For modern features like `std::variant`, `std::optional`, and structured bindings.
5. **Q:** Why integrate Qt6? **A:** For cross-platform file I/O, UI capabilities (if extended), and rich string/collection classes.
6. **Q:** What is a Pager? **A:** A component that manages memory pages and their persistence to disk.
7. **Q:** What eviction policy is used? **A:** LRU (Least Recently Used).
8. **Q:** How are pages structured? **A:** Fixed-size blocks (e.g., 4KB) with headers and payload sections.
9. **Q:** Why use a BTree? **A:** For efficient disk-based data retrieval and range queries.
10. **Q:** What is a B+Tree? **A:** A BTree variant where all values are at the leaf level, optimizing sequential access.
11. **Q:** How does the Evaluator resolve column names? **A:** By consulting the database schema catalog.
12. **Q:** What is a Table Heap? **A:** The storage structure for unordered row data.
13. **Q:** How are transactions handled? **A:** Through a Write-Ahead Log (WAL) and locking (future enhancement).
14. **Q:** What is ACID? **A:** Atomicity, Consistency, Isolation, Durability.
15. **Q:** How do you handle string types in memory? **A:** Stored as variable-length data with offsets.
16. **Q:** What happens when a BTree node gets full? **A:** It splits into two nodes and promotes a median key to the parent.
17. **Q:** How is concurrency managed in the Pager? **A:** Using latches (mutexes/read-write locks) on pages.
18. **Q:** Describe the `Insert` execution pipeline. **A:** Parse -> Validate Schema -> Locate BTree Leaf -> Insert -> Write Page -> Update Metadata.
19. **Q:** What is a Tuple? **A:** A single row/record in a table.
20. **Q:** How are schema changes (DDL) managed? **A:** By updating system catalog tables.
21. **Q:** What is the role of `std::variant`? **A:** To represent dynamically typed SQL values (int, string, float).
22. **Q:** How do you optimize `SELECT *`? **A:** By fetching complete tuples directly from the leaf nodes.
23. **Q:** What is query optimization? **A:** Transforming an AST into an efficient physical execution plan.
24. **Q:** How is sequential scan implemented? **A:** By iterating through all leaf nodes of the B+Tree.
25. **Q:** What is index scan? **A:** Traversing the BTree to find specific keys instead of scanning the whole table.
26. **Q:** How do you handle large data sets exceeding RAM? **A:** The Pager evicts unpinned pages to disk.
27. **Q:** What is a dirty page? **A:** A page modified in memory but not yet written to disk.
28. **Q:** How do you ensure durability? **A:** By flushing dirty pages to disk on commit.
29. **Q:** What is a primary key constraint? **A:** Ensures uniqueness and non-nullability, often backed by a unique BTree index.
30. **Q:** How is a `WHERE` clause evaluated? **A:** By interpreting the binary expression AST node against each tuple.

31. **Q:** What design pattern is used in the Executor? **A:** The Volcano/Iterator model (`Next()`, `Init()`).
32. **Q:** How do you handle NULL values? **A:** Through a null bitmap in the tuple header.
33. **Q:** What is a buffer pool? **A:** The allocated memory space managed by the Pager.
34. **Q:** How are database files organized? **A:** As a sequence of uniform-sized pages.
35. **Q:** What is a slotted page architecture? **A:** A page layout where a directory at the end of the page points to variable-length records.
36. **Q:** How do you handle file growth? **A:** The Pager extends the file by writing zeroed pages.
37. **Q:** Why not use standard file streams for every read/write? **A:** It causes excessive I/O overhead.
38. **Q:** What is a memory leak? **A:** Failing to free allocated memory. Addressed using smart pointers and RAII.
39. **Q:** How do you represent database schemas? **A:** Using a struct/class containing column names, types, and constraints.
40. **Q:** What is a foreign key? **A:** A constraint ensuring referential integrity between tables.
41. **Q:** How do you parse JOIN clauses? **A:** By recognizing the JOIN keyword and building a JoinNode in the AST.
42. **Q:** How is a nested loop join executed? **A:** By iterating over the outer table and matching rows in the inner table.
43. **Q:** What are the benefits of Qt6's QString? **A:** Efficient Unicode handling and copy-on-write semantics.
44. **Q:** How do you unit test the database? **A:** Using frameworks like QTest or Google Test on isolated components.
45. **Q:** What is serialization? **A:** Converting in-memory objects (tuples) into byte arrays for disk storage.
46. **Q:** How do you handle endianness? **A:** By converting integers to a standard byte order before disk writing.
47. **Q:** What is an execution context? **A:** State passed through the executor pipeline (transaction ID, parameters).
48. **Q:** How do you prevent SQL injection? **A:** Through parameterized queries and prepared statements.
49. **Q:** What is the critical path in the database? **A:** The Pager's read/write operations and buffer pool management.
50. **Q:** How do you profile the database? **A:** Using tools like Valgrind, perf, or Qt Creator's built-in profilers.

6. Conclusion

The MiniSQL project demonstrates advanced systems programming techniques using C++17 and Qt6. By building the parser, execution engine, and BTree-backed storage from scratch, it highlights deep architectural knowledge of relational databases.